

Kafka KRaft Dynamic Controller Quorum — Operations & Recovery Runbook

1. Scope

This runbook covers operational management of a **KIP-853 dynamic KRaft controller quorum**.

Example architecture:

```
DC1
  C101  node.id=101
  C102  node.id=102
  C103  node.id=103

DC2
  C201  node.id=201
  C202  node.id=202
```

Normal voter set:

```
101
102
103
201
202
```

Quorum size:

```
N = 5
majority = 3
```

Therefore:

```
2 controllers may fail
and quorum can still operate.

3 controllers unavailable
= quorum majority lost.
```

The runbook distinguishes two fundamentally different classes of failure:

A. MAJORITY EXISTS

-> normal supported Kafka quorum operations

B. MAJORITY LOST

-> STOP normal membership operations

-> preserve surviving metadata

-> forensic / disaster-recovery procedure

2. Important KRaft state

The operator must understand four different pieces of state.

meta.properties

- |
- +--> cluster.id
- +--> node.id
- +--> directory.id
- |
- +--> local storage identity

quorum-state

- |
- +--> LeaderId
- +--> LeaderEpoch
- +--> vote/election state
- |
- +--> local persistent Raft state

__cluster_metadata-0

- |
- +--> snapshots
- +--> log segments
- +--> VotersRecord
- |
- +--> replicated KRaft state

controller.quorum.bootstrap.servers

- |
- +--> addresses used to FIND quorum
- |
- +--> NOT membership

For a dynamic quorum:

```
controller.quorum.bootstrap.servers=  
c101.dc1.example.com:9093,  
c102.dc1.example.com:9093,  
c103.dc1.example.com:9093,  
c201.dc2.example.com:9093,  
c202.dc2.example.com:9093
```

Do **not** define dynamic membership with:

```
controller.quorum.voters=...
```

Dynamic membership comes from replicated KRaft state.

3. Command environment

For the examples below:

```
export KAFKA_HOME=/opt/kafka  
export PATH="$KAFKA_HOME/bin:$PATH"  
  
export CTRL_PORT=9093  
  
export C101=c101.dc1.example.com  
export C102=c102.dc1.example.com  
export C103=c103.dc1.example.com  
export C201=c201.dc2.example.com  
export C202=c202.dc2.example.com  
  
export BOOTSTRAP_CONTROLLER="${C101}:${CTRL_PORT}"
```

For authenticated clusters:

```
export ADMIN_CONFIG=/etc/kafka/admin.properties
```

Example helper:

```
KMQ="$KAFKA_HOME/bin/kafka-metadata-quorum.sh"  
KST="$KAFKA_HOME/bin/kafka-storage.sh"  
KFEAT="$KAFKA_HOME/bin/kafka-features.sh"  
KDUMP="$KAFKA_HOME/bin/kafka-dump-log.sh"
```

If authentication is required, commands generally become:

```
$KMQ
--command-config "$ADMIN_CONFIG"
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --status
```

4. First command during every controller incident

Always begin with:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --status
```

Expected type of output:

```
ClusterId:      ...
LeaderId:       101
LeaderEpoch:   82
HighWatermark:  421304

CurrentVoters:
  101 / UUID101
  102 / UUID102
  103 / UUID103
  201 / UUID201
  202 / UUID202
```

Record at minimum:

```
ClusterId
LeaderId
LeaderEpoch
HighWatermark
CurrentVoters
directoryId of each voter
```

Do not begin changing membership before you have captured this information.

5. Check detailed replication state

Run:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --replication
```

Conceptual output:

NodeId	DirectoryId	LogEndOffset	Lag	LastFetchTimestamp ...
101	UUID101	421305	0	
102	UUID102	421305	0	
103	UUID103	421305	0	
201	UUID201	421305	0	
202	UUID202	421305	0	

This command answers:

Which controller is leader?

Which replicas are caught up?

What is each LogEndOffset?

What is each replica's directory.id?

Which replicas are lagging?

Which node is currently an observer rather than voter?

6. Verify that this is a dynamic quorum

Run:

```
$KFEAT
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe
```

Look for:

```
Feature: kraft.version  
FinalizedVersionLevel: 1
```

Interpretation:

```
kraft.version absent or 0  
  -> static KRaft quorum  
  
kraft.version >= 1  
  -> KIP-853 dynamic quorum
```

If the cluster is static, **do not use the dynamic membership procedures in this runbook.**

7. Daily quorum health check

A simple operational check:

```
#!/usr/bin/env bash  
set -euo pipefail  
  
BOOTSTRAP_CONTROLLER="${1:-c101.dc1.example.com:9093}"  
  
echo "=== KRaft quorum status ==="  
  
kafka-metadata-quorum.sh  
  --bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
  describe --status  
  
echo  
  
echo "=== KRaft replication ==="  
  
kafka-metadata-quorum.sh  
  --bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
  describe --replication
```

A healthy five-controller quorum should normally show:

```
LeaderId != -1  
  
CurrentVoters = 5 controllers  
  
at least 3 controllers reachable  
  
small or zero follower lag
```

```
recent LastFetchTimestamp
```

```
recent LastCaughtUpTimestamp
```

8. Important difference: LEO versus High Watermark

Never treat:

```
LogEndOffset
```

as equivalent to:

```
HighWatermark
```

Example:

```
HighWatermark = 10000
```

```
C201 LEO = 10000
```

```
C202 LEO = 10004
```

This does **not** automatically mean C202 is more authoritative.

The tail:

```
10000 ... 10004
```

may contain records which have not become committed.

Therefore:

```
largest LogEndOffset
```

```
!=
```

```
latest proven committed metadata
```

This distinction becomes critical in disaster recovery.

9. Identify a controller's storage identity

On the controller host:

```
grep -E '^(cluster.id|node.id|directory.id|version)='  
/var/lib/kafka/kraft-metadata/meta.properties
```

Example:

```
cluster.id=q1Sh-9_ISia_zwGINzRvyQ  
node.id=201  
directory.id=AbCdEfGhIjKlMnOpQrStUv  
version=1
```

Record it:

```
CLUSTER_ID="$(grep '^cluster.id='  
/var/lib/kafka/kraft-metadata/meta.properties |  
cut -d= -f2)"  
  
NODE_ID="$(grep '^node.id='  
/var/lib/kafka/kraft-metadata/meta.properties |  
cut -d= -f2)"  
  
DIRECTORY_ID="$(grep '^directory.id='  
/var/lib/kafka/kraft-metadata/meta.properties |  
cut -d= -f2)"
```

Display:

```
printf 'cluster.id=%s\nnode.id=%s\ndirectory.id=%s\n'  
"$CLUSTER_ID"  
"$NODE_ID"  
"$DIRECTORY_ID"
```

Remember:

```
controller replica identity  
≈  
(node.id, directory.id)
```

Example:


```
old C201:
```

```
(201, AAAA)
```

```
newly formatted C201:
```

```
(201, BBBB)
```

These are different storage incarnations.

10. Locate the metadata directory

Normally:

```
metadata.log.dir=/var/lib/kafka/kraft-metadata
```

Check configuration:

```
grep '^metadata.log.dir=' /etc/kafka/controller.properties
```

Then:

```
META_DIR=/var/lib/kafka/kraft-metadata
```

Metadata partition:

```
ls -lah "$META_DIR/__cluster_metadata-0/"
```

You should see files such as:

```
00000000000000000000000000000000.log
...
xxxxxxxxxxxxxxxxxxxx-yyyyyyyyyy.checkpoint

quorum-state
leader-epoch-checkpoint
...
```

Exact files vary by Kafka version and runtime history.

11. Inspect `quorum-state`

First preserve the file:

```
cp
"$META_DIR/__cluster_metadata-0/quorum-state"
"/tmp/quorum-state.$(date +%Y%m%d-%H%M%S) "
```

For routine operations, do **not edit this file**.

It contains local Raft election state.

Conceptually:

```
LeaderId
LeaderEpoch
VotedId
VotedDirectoryId
```

It is **not** the authoritative dynamic voter configuration.

Never fix membership by manually editing:

```
quorum-state
```

12. Decode the KRaft metadata log

List segments:

```
ls -l
"$META_DIR/__cluster_metadata-0/"*.log
```

Decode a segment:

```
$KDUMP
--cluster-metadata-decoder
--files
"$META_DIR/__cluster_metadata-0/00000000000000000000.log"
```

Search for voter-related records:

```
$KDUMP
--cluster-metadata-decoder
--files
"$META_DIR/__cluster_metadata-0/00000000000000000000.log"
| grep -i -E 'VotersRecord|KRaftVersionRecord'
```

For several log files:

```
for f in "$META_DIR/__cluster_metadata-0/*.log; do
    echo "=====$f===="

    $KDUMP
    --cluster-metadata-decoder
    --files "$f"
    | grep -i -E 'VotersRecord|KRaftVersionRecord'
done
```

13. Decode a metadata snapshot

Find snapshots:

```
find "$META_DIR/__cluster_metadata-0"
-maxdepth 1
-name '*.checkpoint'
-printf '%f\n'
| sort
```

Then decode an actual metadata snapshot:

```
$KDUMP
--cluster-metadata-decoder
--files
"$META_DIR/__cluster_metadata-0/<snapshot>.checkpoint"
```

Search specifically for voter state:

```
$KDUMP
--cluster-metadata-decoder
--files
"$META_DIR/__cluster_metadata-0/<snapshot>.checkpoint"
| grep -i -A20 -B5 'VotersRecord'
```

The correct model is:

```
latest valid snapshot
+
records after the snapshot
=
local reconstructed KRaft state
```

Do not assume a historic `VotersRecord` must still exist in an old `.log` file.

14. NORMAL FAILURE — one controller down

Example:

```
C202 down

alive:
101
102
103
201

quorum:
4 / 5

majority:
YES
```

First:

```
$KMQ
--bootstrap-controller "${C101}:9093"
describe --status
```

Then:

```
$KMQ
--bootstrap-controller "${C101}:9093"
describe --replication
```

If C202 merely crashed but its storage remains intact:

```
systemctl status kafka-controller
```

or:

```
ps -ef | grep '[k]afka'
```

Check logs:

```
journalctl  
-u kafka-controller  
-n 200  
--no-pager
```

Restart:

```
sudo systemctl restart kafka-controller
```

Then monitor:

```
watch -n 2  
"$KMQ --bootstrap-controller ${C101}:9093 describe --replication"
```

Expected:

```
C202 starts  
  
C202 uses existing:  
  
node.id=202  
directory.id=UUID202  
  
C202 reconnects  
  
C202 catches up  
  
C202 remains same voter identity
```

No `remove-controller` is required for an ordinary restart.

15. Controller lagging badly

Check:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --replication
```

Pay attention to:

```
LogEndOffset
Lag
LastFetchTimestamp
LastCaughtUpTimestamp
```

Example:

```
Leader C101 LEO = 900000

C201 LEO = 900000
C202 LEO = 650000
```

Investigate:

```
journalctl
-u kafka-controller
--since "30 min ago"
--no-pager
```

Connectivity:

```
nc -vz "$C101" 9093
nc -vz "$C102" 9093
nc -vz "$C103" 9093
```

DNS:

```
getent hosts "$C101"
getent hosts "$C102"
getent hosts "$C103"
```

TLS if applicable:

```
openssl s_client
-connect "${C101}:9093"
-servername "$C101"
```

Do **not** remove a lagging voter as your first action.

First determine whether it can catch up.

16. Add a completely new controller C203

Suppose we want:

```
existing:

101
102
103
201
202

new controller:

203
```

Configuration on C203:

```
process.roles=controller

node.id=203

controller.listener.names=CONTROLLER

listeners=CONTROLLER://c203.dc2.example.com:9093

controller.quorum.bootstrap.servers=
c101.dc1.example.com:9093,
c102.dc1.example.com:9093,
c103.dc1.example.com:9093,
c201.dc2.example.com:9093,
c202.dc2.example.com:9093
```

No:

```
controller.quorum.voters=
```

must be present.

17. Obtain the existing cluster ID

From running quorum:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --status
```

Record:

```
ClusterId
```

For example:

```
export CLUSTER_ID='q1Sh-9_ISia_zwGINzRvyQ'
```

Never generate a new cluster ID for a controller joining an existing cluster.

Wrong:

```
kafka-storage.sh random-uuid
```

for this purpose.

Correct:

```
reuse the existing Kafka ClusterId
```

18. Format the new controller as NOT an initial voter

On C203:

```
$KST format  
--cluster-id "$CLUSTER_ID"  
--config /etc/kafka/controller.properties  
--no-initial-controllers
```

This distinction is critical.

Do **not** use:

```
--standalone
```

for a controller joining an existing quorum.

Do **not** construct a new independent voter set.

Use:

```
--no-initial-controllers
```

19. Verify C203 identity

```
grep -E  
  '^(cluster.id|node.id|directory.id|version)='  
  /var/lib/kafka/kraft-metadata/meta.properties
```

Expected:

```
cluster.id=<existing cluster ID>  
node.id=203  
directory.id=<new UUID>
```

20. Start C203

```
sudo systemctl start kafka-controller
```

Check:

```
sudo systemctl status kafka-controller
```

Logs:

```
journalctl  
  -u kafka-controller  
  -f
```

C203 should initially participate in metadata replication without yet being part of the committed voter set.

21. Verify C203 is caught up before promotion

Run:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --replication
```

You should see C203 represented as an observer/non-voting replica while it catches up.

Wait for approximately:

```
C203 LogEndOffset ~= leader LogEndOffset

Lag ~= 0

LastCaughtUpTimestamp recent
```

Do not add C203 as voter while badly behind.

22. Add C203 to the voter set

With current stable Kafka CLI form:

```
$KMQ
--command-config /etc/kafka/controller-203.properties
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
add-controller
```

The supplied controller config allows the command to identify the new controller's:

```
node.id
directory.id
controller endpoints
```

Afterward:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --status
```

Expected:

```
CurrentVoters:
```

```
101
102
103
201
202
203
```

Then:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --replication
```

23. Temporary six-controller quorum

During replacement you may temporarily have:

```
101
102
103
201
202
203
```

That means:

```
N = 6
majority = 4
```

Do not deliberately leave the cluster at six voters unless there is an architectural reason.

Typical replacement:

```
5 voters
  |
  +--> add replacement
  |
6 voters
  |
  +--> remove old controller
  |
5 voters
```

24. Planned removal of a healthy controller

Example: remove C202.

First obtain authoritative voter identity:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --status
```

Record:

```
controller-id = 202

controller-directory-id = UUID202
```

Recheck replication:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --replication
```

Make sure removing C202 will still leave a healthy majority.

Then remove:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
remove-controller
--controller-id 202
--controller-directory-id UUID202
```

Verify:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --status
```

Expected:

```
CurrentVoters:
```

```
101  
102  
103  
201
```

Only after membership removal has succeeded:

```
sudo systemctl stop kafka-controller
```

Do not simply stop/delete a controller and assume dynamic membership disappears automatically.

25. Replace failed C202 storage

Scenario:

```
old:  
  
node.id=202  
directory.id=AAAA  
  
disk lost
```

The other four controllers are healthy.

First confirm majority:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --status
```

Then:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --replication
```

Record the old identity:

```
202 + AAAA
```

26. Build new C202 storage

New storage keeps:

```
node.id=202
```

but gets a new:

```
directory.id=BBBB
```

Format:

```
$KST format
--cluster-id "$CLUSTER_ID"
--config /etc/kafka/controller.properties
--no-initial-controllers
```

Verify:

```
grep -E
'^(cluster.id|node.id|directory.id)= '
/var/lib/kafka/kraft-metadata/meta.properties
```

Expected:

```
cluster.id=CLUSTER-A
node.id=202
directory.id=BBBB
```

27. Start replacement C202 and allow catch-up

```
sudo systemctl start kafka-controller
```

Monitor:

```
watch -n 2  
"$KMQ --bootstrap-controller $BOOTSTRAP_CONTROLLER describe --replication"
```

Conceptually:

```
old voter:  
  202 + AAAA  
  
replacement observer:  
  202 + BBBB
```

Kafka can distinguish them because the directory IDs differ.

28. Add replacement voter identity

Once caught up:

```
$KMQ  
--command-config /etc/kafka/controller.properties  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
add-controller
```

Then check:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --status
```

Inspect carefully which `(node.id, directory.id)` identities are present.

29. Remove old storage identity

Remove:

```
202 + AAAA
```

not:

```
202 + BBBB
```

Command:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
remove-controller
--controller-id 202
--controller-directory-id AAAA
```

Verify:

```
$KMQ
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"
describe --status
```

Final desired state:

```
202 + BBBB
```

30. Never select a controller only by `node.id`

Dangerous:

```
remove controller 202
```

without understanding the directory identity.

Always think:

```
VOTER IDENTITY

node.id
+
directory.id
```


This is particularly important during disk replacement.

31. Controller has wrong `cluster.id`

Check:

```
grep '^cluster.id='  
/var/lib/kafka/kraft-metadata/meta.properties
```

Compare with:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --status
```

If:

```
local cluster.id != active ClusterId
```

STOP.

Do not modify `meta.properties` manually.

Do not copy another controller's `meta.properties`.

Do not try to force the controller into the cluster by changing:

```
cluster.id
```

The storage was formatted for another cluster or incorrectly provisioned.

Re-provision the controller correctly.

32. Controller has wrong `node.id`

Compare:

```
grep '^node.id='  
/var/lib/kafka/kraft-metadata/meta.properties
```

with:

```
grep '^node.id=' /etc/kafka/controller.properties
```

Example problematic condition:

```
config:  
node.id=201  
  
meta.properties:  
node.id=202
```

STOP.

Do not edit `meta.properties` manually.

Investigate why the wrong metadata directory or configuration was mounted.

33. `directory.id` changed unexpectedly

Compare current voter set:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --status
```

with local:

```
grep '^directory.id='  
"$META_DIR/meta.properties"
```

If voter set contains:

```
201 + AAAA
```

but local storage says:

```
201 + BBBB
```

you are dealing with a storage replacement, not an ordinary restart.

Follow the replacement procedure.

34. Bootstrap server unreachable

Remember:

```
controller.quorum.bootstrap.servers
=
discovery addresses
```

You do not need the first listed controller specifically.

Try another known controller:

```
$KMQ
--bootstrap-controller "${C201}:9093"
describe --status
```

Then:

```
$KMQ
--bootstrap-controller "${C202}:9093"
describe --status
```

Check connectivity:

```
for h in "$C101" "$C102" "$C103" "$C201" "$C202"; do
  echo "=== $h ==="
  nc -vz "$h" 9093
done
```

A stale or unreachable bootstrap endpoint does not itself alter quorum membership.

35. Leader controller failed

Suppose:

```
LeaderId = 101
```

and C101 fails.

With four other voters alive:

```
102
103
201
202
```

you still have:

```
4/5
```

The remaining quorum should elect a new leader automatically.

Observe:

```
watch -n 1
"$KMQ --bootstrap-controller ${C201}:9093 describe --status"
```

You should eventually see:

```
LeaderId: <new controller>
LeaderEpoch: <higher epoch>
```

Do **not** manually "promote" a controller.

KRaft performs leader election.

There is no normal operator command equivalent to:

```
promote C201 to controller leader
```

36. No leader, but 3/5 controllers appear alive

First check reachability between controllers.

From C101:

```
for h in "$C102" "$C103" "$C201" "$C202"; do
  nc -vz "$h" 9093
done
```

Repeat from the other controller hosts.

Check DNS:

```
getent hosts "$C101"  
getent hosts "$C102"  
getent hosts "$C103"  
getent hosts "$C201"  
getent hosts "$C202"
```

Check system clock:

```
timedatectl
```

Check logs:

```
journalctl  
-u kafka-controller  
--since "15 min ago"  
--no-pager  
| grep -Ei  
'raft|quorum|leader|vote|election|fetch|timeout|error'
```

Check current configuration:

```
grep -E  
'^(node.id|controller.listener.names|listeners|  
controller.quorum.bootstrap.servers|listener.security.protocol.map)='  
/etc/kafka/controller.properties
```

The most common operational categories are:

```
network partition  
  
DNS problem  
  
listener mismatch  
  
TLS/SASL failure  
  
bad controller endpoint  
  
controller process unavailable  
  
severe latency
```

```
storage problem
```

Do not modify membership until connectivity/election problems are understood.

37. Planned controller maintenance

Before taking C202 down:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --status
```

Then:

```
$KMQ  
--bootstrap-controller "$BOOTSTRAP_CONTROLLER"  
describe --replication
```

Verify at least:

```
other 4 voters healthy  
  
lag small  
  
leader stable
```

Then:

```
sudo systemctl stop kafka-controller
```

For temporary maintenance there is no reason to remove C202 from membership.

After maintenance:

```
sudo systemctl start kafka-controller
```

Monitor catch-up:

```
watch -n 2  
"$KMQ --bootstrap-controller $BOOTSTRAP_CONTROLLER describe --replication"
```

38. Do not remove voters for short outages

Bad procedure:

```
controller unavailable for 5 minutes  
|  
v  
remove-controller
```

Normal procedure:

```
controller unavailable temporarily  
|  
v  
repair/restart controller  
|  
v  
same node.id  
same directory.id  
|  
v  
catch up
```

Use membership changes when the membership itself needs to change.

39. DC2 complete failure — C201 and C202 lost

Topology:

```
DC1:  
101  
102  
103  
  
DC2:  
201 X  
202 X
```

Survivors:

3 / 5

Majority survives.

This is a normal quorum-degraded situation.

Check:

```
$KMQ  
--bootstrap-controller "${C101}:9093"  
describe --status
```

Then:

```
$KMQ  
--bootstrap-controller "${C101}:9093"  
describe --replication
```

If DC2 is expected to return, do not remove C201/C202 immediately.

When DC2 returns:

```
C201 starts  
C202 starts  
  
same storage identities  
  
metadata catch-up  
  
quorum returns to 5/5
```

40. Permanent loss of C201 and C202

If DC2 is permanently destroyed but DC1 still has:

```
101  
102  
103
```

you retain quorum.

You can replace C201/C202 using normal dynamic membership procedures.

For each lost controller:

```
provision replacement

format with existing CLUSTER_ID

--no-initial-controllers

start

wait for catch-up

add-controller

remove stale old voter identity
```

Perform one membership change at a time.

41. CRITICAL INCIDENT — DC1 lost

Now the dangerous case:

```
DC1:
101 X
102 X
103 X

DC2:
201
202
```

Current voters:

```
101
102
103
201
202
```

Survivors:

```
2 / 5
```

Required majority:

```
3
```

Therefore:

```
NO QUORUM
```

42. Hard safety gate after majority loss

When only C201 and C202 remain:

DO NOT

```
kafka-storage.sh format ...
```

on C201 or C202.

Do not:

```
kafka-storage.sh format --standalone ...
```

Do not:

```
kafka-storage.sh random-uuid
```

and reinitialize the survivors.

Do not:

```
kafka-metadata-quorum.sh add-controller
```

expecting 2/5 to approve it.

Do not:

```
kafka-metadata-quorum.sh remove-controller
```

expecting the minority to redefine membership.

Do not delete:

```
__cluster_metadata-0
```

Do not delete:

```
quorum-state
```

Do not edit:

```
meta.properties
```

Do not choose the controller with the largest LEO and immediately restart a new quorum.

43. Why normal membership commands stop working

The voter set is still conceptually:

```
101
102
103
201
202
```

A committed membership update requires a valid functioning KRaft consensus process.

But only:

```
201
202
```

remain.

Therefore:

```
2 < majority of 5
```

The minority cannot safely say:

```
"we remove 101, 102 and 103"
```

because those two surviving controllers do not constitute the old quorum majority.

44. First action after majority loss: freeze writes to metadata storage

Stop surviving controllers if necessary to prevent uncontrolled changes during investigation:

```
sudo systemctl stop kafka-controller
```

Verify:

```
ps -ef | grep '[k]afka'
```

The objective is:

```
C201 metadata = preserved exactly
```

```
C202 metadata = preserved exactly
```

45. Make forensic copies

On C201:

```
sudo rsync
-aHAX
--numeric-ids
/var/lib/kafka/kraft-metadata/
/recovery/c201-kraft-metadata/
```

On C202:

```
sudo rsync
-aHAX
--numeric-ids
/var/lib/kafka/kraft-metadata/
/recovery/c202-kraft-metadata/
```

Even better, snapshot the underlying volume if your infrastructure supports it.

Perform investigation on copies.

Do not manipulate the only surviving metadata copies.

46. Hash the forensic copies

C201:

```
find /recovery/c201-kraft-metadata
  -type f
  -print0
  | sort -z
  | xargs -0 sha256sum
> /recovery/c201.sha256
```

C202:

```
find /recovery/c202-kraft-metadata
  -type f
  -print0
  | sort -z
  | xargs -0 sha256sum
> /recovery/c202.sha256
```

This gives you evidence that the copies used for analysis did not change.

47. Record identity of C201

```
cat
  /recovery/c201-kraft-metadata/meta.properties
```

Record:

```
cluster.id
node.id
directory.id
version
```

Expected:

```
node.id=201
directory.id=UUID201
```

48. Record identity of C202

```
cat
/recovery/c202-kraft-metadata/meta.properties
```

Expected:

```
node.id=202
directory.id=UUID202
```

Verify:

```
C201 cluster.id == C202 cluster.id
```

If they differ:

```
STOP
```

You may not even be looking at metadata from the same Kafka cluster.

49. Save quorum-state

C201:

```
cp
/recovery/c201-kraft-metadata/__cluster_metadata-0/quorum-state
/recovery/c201-quorum-state
```

C202:

```
cp
/recovery/c202-kraft-metadata/__cluster_metadata-0/quorum-state
/recovery/c202-quorum-state
```

Treat this as supporting Raft evidence.

Do not treat it as the authoritative voter membership database.

50. List metadata files on both survivors

```
find
  /recovery/c201-kraft-metadata/__cluster_metadata-0
  -maxdepth 1
  -type f
  -printf '%f %s\n'
  | sort
> /recovery/c201-files.txt
```

And:

```
find
  /recovery/c202-kraft-metadata/__cluster_metadata-0
  -maxdepth 1
  -type f
  -printf '%f %s\n'
  | sort
> /recovery/c202-files.txt
```

Compare:

```
diff -u
  /recovery/c201-files.txt
  /recovery/c202-files.txt
```

51. Decode all surviving metadata log segments

C201:

```
mkdir -p /recovery/c201-decoded
```

Then:

```
for f in
  /recovery/c201-kraft-metadata/__cluster_metadata-0/*.log
do
  base="$(basename "$f")"
```

```
"$KDUMP"  
  --cluster-metadata-decoder  
  --files "$f"  
  > "/recovery/c201-decoded/${base}.txt"  
done
```

C202:

```
mkdir -p /recovery/c202-decoded
```

```
for f in  
  /recovery/c202-kraft-metadata/__cluster_metadata-0/*.log  
do  
  base="$(basename "$f")"  
  
  "$KDUMP"  
    --cluster-metadata-decoder  
    --files "$f"  
    > "/recovery/c202-decoded/${base}.txt"  
done
```

52. Find all visible VotersRecord entries

C201:

```
grep  
  -Rni  
  'VotersRecord'  
  /recovery/c201-decoded
```

C202:

```
grep  
  -Rni  
  'VotersRecord'  
  /recovery/c202-decoded
```

Compare the voter history.

For example:

```
C201:
```



```
VotersRecord A
101 102 103 201 202
```

C202:

```
VotersRecord A
101 102 103 201 202
```

or perhaps:

```
C201:
latest visible set =
101 102 103 201 203
```

```
C202:
latest visible set =
101 102 103 201 202
```

The latter requires detailed investigation.

53. Decode snapshots as well

Do **not** inspect only `.log` segments.

List snapshots:

```
find
/recovery/c201-kraft-metadata/__cluster_metadata-0
-name '*.checkpoint'
-printf '%p\n'
| sort
```

For each relevant metadata snapshot:

```
"$KDUMP"
--cluster-metadata-decoder
--files "$SNAPSHOT"
```

Repeat for C202.

Remember:

```
effective local metadata state
=
snapshot
+
records following snapshot
```

54. Compare the surviving histories

You now need to establish:

C201:

```
cluster.id
directory.id
snapshot offset
snapshot epoch
log segment ranges
latest record offset
leader epochs
visible VotersRecords
```

C202:

```
cluster.id
directory.id
snapshot offset
snapshot epoch
log segment ranges
latest record offset
leader epochs
visible VotersRecords
```

Conceptually create:

	C201	C202
ClusterId	A	A
DirectoryId	UUID201	UUID202
Snapshot	01/E1	02/E2
Last log	LE01	LE02
Voters	...	...

Epoch

55. Do not use this unsafe rule

Never use:

```
if C201 LE0 > C202 LE0
then C201 = safe seed
```

This is unsafe.

Possible situation:

```
C201:

LE0 = 5005

C202:

LE0 = 5000

original HighWatermark:
5000
```

Offsets:

```
5001-5005
```

on C201 may have been uncommitted.

56. There is another opposite danger

Suppose both surviving DC2 controllers have:

```
LE0 = 10000
```

but before DC1 failed the three DC1 controllers committed:

10001
10002
10003

to:

C101
C102
C103

Those three constituted a majority:

3 / 5

Therefore those records could have been committed even if C201 and C202 never received them.

After DC1 disappears:

C201 and C202 cannot prove
that offset 10000 was the final committed offset.

This is why minority recovery cannot guarantee RPO=0 solely from surviving minority logs.

57. Majority-loss decision

The operational question is not:

Which survivor has the biggest log?

It is:

Can we prove the last metadata record
that was committed by the original quorum?

If:

YES

you may be able to construct a controlled recovery process around that known state.

If:

NO

then recovery must explicitly acknowledge possible metadata loss.

58. Why `--standalone` is not a routine DR command

This command:

```
kafka-storage.sh format
  --cluster-id "$CLUSTER_ID"
  --standalone
  --config controller.properties
```

creates a new bootstrap snapshot and establishes the formatted controller as a single-voter quorum.

That makes it appropriate for **initial cluster bootstrapping**.

It is not a normal command for:

```
"three of five controllers disappeared,
so make one survivor leader."
```

Formatting surviving metadata would overwrite/reinitialize critical state and can destroy evidence required to determine what was committed.

Therefore it is forbidden in the initial majority-loss response.

59. `controller.quorum.bootstrap.servers` cannot repair majority loss

Changing:

```
controller.quorum.bootstrap.servers=
c201.dc2.example.com:9093,
c202.dc2.example.com:9093
```

does not redefine:

```
CurrentVoters
```

from:

```
101 102 103 201 202
```

to:

```
201 202
```

Bootstrap servers answer:

```
WHERE CAN I TRY TO FIND QUORUM?
```

They do not answer:

```
WHO IS ALLOWED TO VOTE?
```

Therefore bootstrap configuration is not an emergency quorum override.

60. Never manually edit VotersRecord

Do not attempt:

```
open metadata log  
  
change voter IDs  
  
remove C101/C102/C103  
  
save file
```

KRaft metadata is an ordered replicated log with checksums, offsets, epochs, snapshots and consensus semantics.

Manual byte-level modification is not an operational membership mechanism.

61. Never copy `meta.properties` between controllers

Bad:

```
scp
  c201:/var/lib/kafka/kraft-metadata/meta.properties
  c202:/var/lib/kafka/kraft-metadata/meta.properties
```

Why:

```
C201 identity:

node.id=201
directory.id=UUID201

C202 identity:

node.id=202
directory.id=UUID202
```

They must remain distinct.

62. Never copy only `quorum-state` to another controller

Bad:

```
cp c201/quorum-state c202/quorum-state
```

`quorum-state` belongs to the local Raft participant.

It is not a portable membership configuration.

63. Do not reconstruct membership from bootstrap configuration

Wrong inference:

```
controller.quorum.bootstrap.servers=  
c201:9093,c202:9093
```

therefore:

```
voters = 201,202
```

This is incorrect.

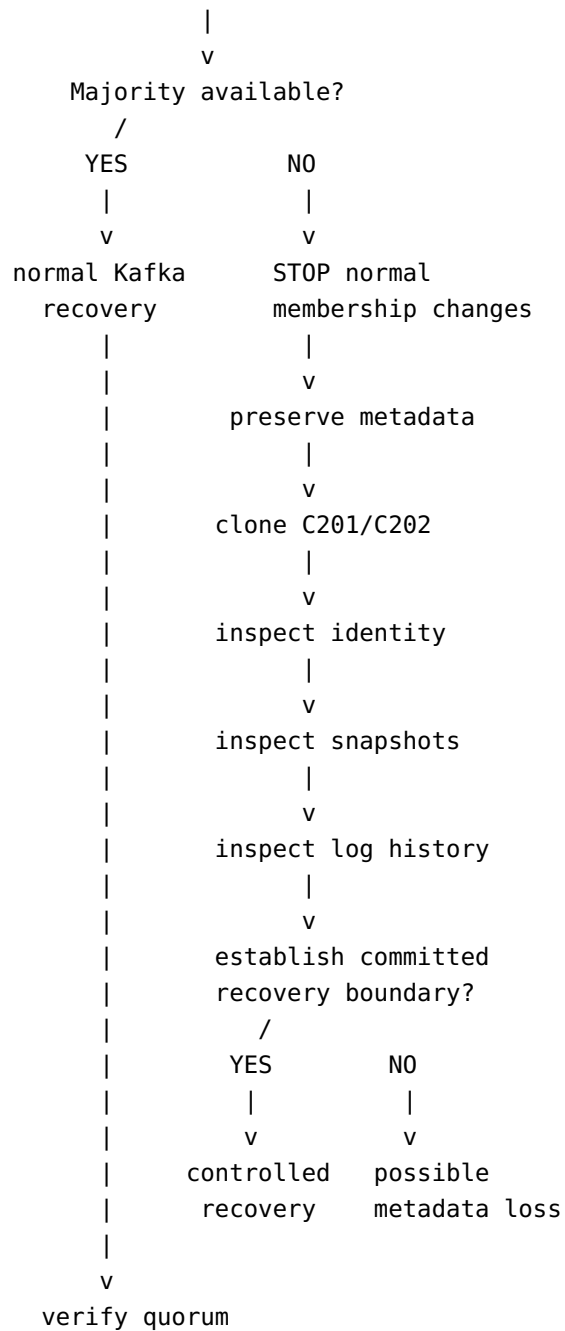
The authoritative dynamic voter state comes from KRaft metadata.

64. Operational command matrix

Operation	Primary command
Quorum summary	<code>kafka-metadata-quorum.sh ... describe --status</code>
Replica state	<code>kafka-metadata-quorum.sh ... describe --replication</code>
Check dynamic quorum feature	<code>kafka-features.sh ... describe</code>
Format new controller joining cluster	<code>kafka-storage.sh format ... --no-initial-controllers</code>
Add voter	<code>kafka-metadata-quorum.sh ... add-controller</code>
Remove voter	<code>kafka-metadata-quorum.sh ... remove-controller</code>
Inspect <code>meta.properties</code>	<code>cat/grep meta.properties</code>
Decode metadata log	<code>kafka-dump-log.sh --cluster-metadata-decoder</code>
Decode snapshot	<code>kafka-dump-log.sh --cluster-metadata-decoder</code>
Inspect metadata image	<code>kafka-metadata-shell.sh --snapshot ...</code>
Temporary controller restart	service manager / <code>kafka-server-start.sh</code>
Majority lost	no normal add/remove command

65. Incident decision tree

```
KRaft controller incident  
|  
v  
describe --status
```

66. Fast incident checklist

Step 1

```
kafka-metadata-quorum.sh
--bootstrap-controller c201.dc2.example.com:9093
describe --status
```

Step 2

```
kafka-metadata-quorum.sh  
  --bootstrap-controller c201.dc2.example.com:9093  
  describe --replication
```

Step 3

Determine:

```
How many voters?  
  
How many alive?  
  
Is LeaderId known?  
  
What is LeaderEpoch?  
  
What is HighWatermark?  
  
Which directory IDs are voters?
```

Step 4

If majority exists:

```
repair  
restart  
catch up  
add/remove/replace if necessary
```

Step 5

If majority does not exist:

```
DO NOT FORMAT  
  
DO NOT CHANGE MEMBERSHIP  
  
DO NOT DELETE METADATA  
  
STOP SURVIVORS IF NEEDED  
  
COPY METADATA  
  
HASH COPIES
```

```
COMPARE C201/C202

DECODE SNAPSHOTS

DECODE LOGS

IDENTIFY VotersRecord HISTORY

DETERMINE WHETHER COMMITTED
RECOVERY BOUNDARY IS PROVABLE
```

67. Commands I would put on the first page of the production runbook

1. Quorum status

```
bin/kafka-metadata-quorum.sh
--bootstrap-controller c101.dc1.example.com:9093
describe --status
```

2. Replication status

```
bin/kafka-metadata-quorum.sh
--bootstrap-controller c101.dc1.example.com:9093
describe --replication
```

3. Verify KIP-853 dynamic quorum

```
bin/kafka-features.sh
--bootstrap-controller c101.dc1.example.com:9093
describe
```

4. Inspect controller identity

```
grep -E
'^((cluster.id|node.id|directory.id|version))='
/var/lib/kafka/kraft-metadata/meta.properties
```

5. Decode metadata segment

```
bin/kafka-dump-log.sh
--cluster-metadata-decoder
```

```
--files  
/var/lib/kafka/kraft-metadata/__cluster_metadata-0/<segment>.log
```

6. Decode metadata snapshot

```
bin/kafka-dump-log.sh  
  --cluster-metadata-decoder  
  --files  
  /var/lib/kafka/kraft-metadata/__cluster_metadata-0/<snapshot>.checkpoint
```

7. Format NEW controller joining existing cluster

```
bin/kafka-storage.sh format  
  --cluster-id "$CLUSTER_ID"  
  --config /etc/kafka/controller.properties  
  --no-initial-controllers
```

8. Add caught-up controller

```
bin/kafka-metadata-quorum.sh  
  --command-config /etc/kafka/controller.properties  
  --bootstrap-controller c101.dcl.example.com:9093  
add-controller
```

9. Remove voter

```
bin/kafka-metadata-quorum.sh  
  --bootstrap-controller c101.dcl.example.com:9093  
remove-controller  
  --controller-id 202  
  --controller-directory-id "$DIRECTORY_ID"
```

68. Golden rule

The safest operational model is:

```
NORMAL OPERATIONS  
  
quorum majority still exists  
  |  
  v  
Kafka consensus decides  
  |
```

```
      v
add/remove voter safely

      DISASTER RECOVERY

quorum majority is lost
  |
  v
Kafka consensus cannot decide
  |
  v
preserve surviving state
  |
  v
determine what was committed
  |
  v
only then design reconstruction
```

And most importantly:

```
controller.quorum.bootstrap.servers
does not recreate quorum.

node.id alone
does not identify a voter incarnation.

largest LogEndOffset
does not prove committed authority.

quorum-state
does not define dynamic membership.

VotersRecord / snapshot state
must be interpreted together with
Raft commit semantics.
```